

UPAY SHUROKKHA

Safer payments and smarter agent liquidity

AI Hackathon 2026 | DIU CPC x upay

Independent prototype report | Synthetic data only

The problem

An MFS agent can hold enough total working capital but lack physical cash for cash-out or electronic float for cash-in. Customers also face payment requests whose purpose may be misleading. This prototype supports two decisions: what an agent should rebalance, and what a customer should verify before sending.

Implemented solution

A Streamlit app forecasts hourly requested demand, recommends one feasible exchange, and compares actual simulated outcomes with baselines. A separate payment check uses a multilingual text classifier and transparent contextual rules to produce a concern index, explain signals and record a simulated customer/reviewer action.

Measured evidence

Measure	Observed result
Cash-out forecasting MAE	1,613.86 BDT, versus 2,274.67 for historical average
Mean net earnings	339.88 BDT/agent-day; historical strategy 328.89; no action 334.65
Text classifier holdout	11 true alerts, 2 false alerts, 1 miss, 10 correct non-alerts

These results describe one synthetic generator and a tiny authored language dataset. They do not establish real fraud prevention, production forecast quality or commercial returns.

Scope and provenance

Primary track: Merchant & Agent Intelligence. Supporting capability: Trust & Risk Intelligence. The public repository is github.com/Rawnak-Alam/upay-shurokkha. The team will supply the verified live deployment and demonstration video through the submission portal. No user interviews or real-world pilot were conducted.

1. Product decisions and workflows

Product grounding: the public upay Agent app listing describes a home dashboard, real-time statement, item-wise transactions and transparent commissions. The actual data schema, rates and account APIs are not disclosed. This demo therefore uses invented transaction history, a manually entered cash count and a fictional wallet balance.

<https://play.google.com/store/apps/details?id=bd.com.upay.agent>

Agent planning

On a phone-width interface, select a fictional agent and example day; enter physical cash and electronic float. The app forecasts cash-in/out separately, estimates a shortage window, and compares feasible exchanges at 09:00, 12:00 or 15:00, including doing nothing. Illustrative costs and capacity are built into the example and disclosed as simulation assumptions. Revealing actual demand compares strategies on identical requested tickets and arrival order.

Customer action	Physical cash	Electronic float
Cash-in 1,000 BDT	+1,000 BDT	-1,000 BDT
Cash-out 1,000 BDT	-1,000 BDT	+1,000 BDT

The simulator serves or refuses each complete ticket. Cash plus float is conserved. A positive proposed cash_delta exchanges float for cash; a negative value does the reverse. A planned exchange that becomes unaffordable under actual demand is skipped and logged. No borrowing or hidden capital injection occurs.

Payment safety

The customer provides an amount and optional English, Banglish or Bangla description. The visible demo uses fictional prior-recipient and typical-amount history only when that history exists; account recovery, device change and reviewed cases are not supplied. The app explains contributing signals, offers cancellation or review, and requires independent-verification confirmation before simulated continuation.

Human review

The case view preserves input evidence, contributions, customer choice and reviewer disposition within the browser session. It supports requesting evidence or escalation; it does not determine criminality. No external complaint or payment is sent.

Important interpretation

A score of 73/100 in a default example means high concern under this design. It does not mean a 73% probability of fraud. Missing history can yield Insufficient information. Identity, age or a lack of complaints cannot establish that an account is safe.

2. Data, models and decision logic

Synthetic requested demand

The seeded generator creates 154 days, three fictional agents and 12 hourly bins per day: 5,544 rows. Assumed patterns include morning cash-in, afternoon cash-out, Friday/Saturday effects, beginning/end-of-month effects, lognormal variation and unannounced afternoon shocks. These patterns were chosen for simulation, not measured from upay. Requested demand includes requests that may be refused.

Forecast training and baseline

Extra Trees uses 100 trees and a minimum leaf size of eight. Inputs are hour, weekday, payday/weekend flags, elapsed day, agent code and same-hour lags at one, seven and 14 days. The historical baseline averages lags seven and 14. Neither uses the selected day's outcomes or the generator's shock flag.

Split	Period	Rows
Training	Jan 15 - Apr 22, 2026	3,528
Validation	Apr 23 - May 13, 2026	756
Test	May 14 - Jun 3, 2026	756

Dates are simulated. Validation sets pooled error radii and three demand multipliers. Models are not refitted on the test. Later test forecasts may use observed earlier-day histories, which is consistent with daily rolling forecasts. Published results use the same test split throughout.

Action selection

The optimizer searches a finite amount grid and three possible hours. It compares three validation-derived demand scenarios with different ticket ordering. Lower cost maximizes expected commission minus exchange cost. Balanced and Higher availability add an explicit service preference for reducing refused value. The search is approximate and does not guarantee a global optimum.

Multilingual text model

Character TF-IDF n-grams of length two to five feed logistic regression. The authored dataset contains 28 scenario families with three language versions each: 48 train, 12 validation, 24 test sentences. Translations stay together within one split. Validation selects a threshold penalizing false warnings twice as much as misses. The full concern index then adds disclosed contextual rules; its weights are not learned or calibrated.

3. Forecasting and agent economics

Hourly MAE (BDT)	Historical average	Extra Trees
Cash-in	1,466.07	1,209.83
Cash-out	2,274.67	1,613.86

The empirical ranges achieved 87.6% test coverage for cash-in and 91.3% for cash-out. These are pooled marginal results; full-day simultaneous coverage or guaranteed coverage on another distribution is not claimed.

Identical demand and capital for all strategies

Strategy	Net BDT/day	Refused requests/day	Exchange BDT/day
No rebalance	334.65	7.95	0.00
Fixed 50/50 reserve	316.36	5.59	34.00
Historical + optimizer	328.89	7.92	6.35
ML + optimizer	339.88	5.06	13.41

Means cover 63 synthetic agent-days, each starting with cash 6,000 BDT and float 24,000 BDT. The ML strategy gained 10.99 BDT per day over historical forecasting and 5.22 over no rebalancing. It beat the historical strategy on 24 days, lost on four, and tied on 35. Two ML exchanges became infeasible under realized demand and were skipped.

Economic assumptions

Assumed commission is 0.4% of served value. An executed exchange costs 25 BDT plus 0.1% of amount; capacity is 20,000 BDT. Fees and commission use a separate operating ledger. These are invented rates, not upay terms. Rent, financing, travel and overhead are excluded. Expected net change is not a guaranteed increase in total business profit.

Stress cases

High exchange costs select no action. With insufficient total capital, many requests remain unserved. Unannounced cash-out surges reduce coverage. One representative last-day stress replay still refused 117 requests despite an exchange. This demonstrates limits of both capital and forecast information; it is not a claim that every shock can be handled.

4. Safety results and responsible use

24-sentence text test	ML classifier	Keyword baseline
True alerts / missed scams	11 / 1	3 / 9
False alerts / correct non-alerts	2 / 10	3 / 9
Precision	84.6%	50.0%
Recall	91.7%	25.0%
False-warning rate	16.7%	25.0%

The test contains 12 scam and 12 legitimate sentences. It is artificially balanced and cannot establish production precision at real fraud prevalence. These metrics evaluate only the text classifier, not the combined concern index or effectiveness of customer interventions.

Language and failure analysis

Each language has only eight test sentences. Bangla: one false alert, no miss. Banglish: one false alert and one miss. English: no error in this tiny set. This is not sufficient evidence of fairness or language coverage. Negated or quoted scam narratives can trigger warnings, while new paraphrases may be missed. The app exposes the actual test mistakes.

Transparent policy

In the simplified customer-facing demo, reviewed-case, device and recovery fields are all unavailable and never supplied as evidence. The visible examples use only the entered amount and reason plus fictional sender history. The hospital example can be falsely flagged and is explained as such.

The text contribution is capped at 60. A first recipient adds eight; an unusually large amount adds up to 12; recovery plus a new device adds 22; reviewed adverse cases add up to 30. Total is capped at 100. Medium begins at 35 and high at 65. All weights are design assumptions. Details and formulas are documented in the model card.

Privacy, security and oversight

No real customer records or external language API are used. Text is processed locally in the app process and cases stay in session memory, with optional evidence download. This demo has no login, role enforcement or permanent audit trail, so public testing must use fictional examples. Production requires governed data access, authentication, retention controls, input limits, monitoring and approved policy for consequential actions.

Adversarial and missing-data behaviour

Descriptions are classified as text, never executed as code or used as agent instructions. Empty descriptions do not invent language evidence. Missing history is explicit and suppresses prior-recipient and typical-amount rules. Adversarial wording remains an unresolved modeling risk. No real review or complaint evidence is used.

5. Reproduction and next steps

Run and inspect

On Windows, open the repository folder and run `setup_windows.cmd`, then `start_windows.cmd`. The app opens at `localhost:8501`. No API key is required. Run the virtual environment's Python with `-m pytest -q` for tests and `scripts/evaluate.py` to rebuild data and metrics. Sixteen automated tests cover accounting, rejected requests, constraints, data leakage and the main UI flows.

Deployment and submission

Deploy `app.py` from the public repository on Streamlit Community Cloud with Python 3.12 and `requirements.txt`. Verify the real public URL, insert it into `README` and `submission.json`, and record the demonstration. The user must provide both registered member names, team identity, video access and portal confirmation. A five-to-six-minute narration script and validation checklist are included.

Controlled validation proposal

Begin with governed, anonymized operational data in shadow mode. Collect refused-demand observations as well as completed transactions. Compare forecasting and recommendations with existing operational practice using full costs and partner constraints. For safety, use independently reviewed outcomes, evaluate new-account and language cohorts, and test whether warnings improve comprehension without excessive disruption.

Phase 2 adaptation

Data generation, model inference, accounting, safety policy and UI are separate modules. New organizer requirements can change capital limits, costs, partner availability, forecast updates or review logic. Each change should be tested and committed during the on-site period. No particular Phase 2 task or production access is assumed.

Resources and disclosure

Implementation: Python, Streamlit, Plotly, NumPy, pandas, scikit-learn and ReportLab. AI-assisted code and draft development used OpenAI Codex. The project uses no third-party customer dataset or pretrained language model. The team must validate the implementation and be able to explain it. Existing repository MIT license is retained.

References: supplied AI Hackathon 2026 Student Project Guideline, DIU CPC x upay; supplied AI DEV FEST 2026 Official Rulebook. Deployment documentation: <https://docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/deploy>. Numeric evidence: `artifacts/metrics.json`, `artifacts/liquidity_replay.csv` and `artifacts/safety_holdout.csv`.